

Seen

Assignment - 03

(22BCON1068) DSA (sheet-05)

DATE: / /

PAGE NO.: 44

Q. • Set-A ($2 \times 5 = 10$)

Q1. What is Linked List and write advantages of Linked List over Array.

Ans. A Linked List is a linear data structure that stores a sequence of elements. Each element in a Linked List is called a node; and each node contains a value and a pointer to the next node in the sequence. The first node in a linked list is called the head node, and the last node is called tail node.

→ Advantages of Linked List over Array:-

- Efficient insertion and deletion.
- Dynamic size.
- Memory efficiency.
- Easy implementation of abstract data types.
- More efficient sorting in some cases.

Q2. What is Queue and write applications of Queue.

Ans. A queue is a linear data structure that follows the first-in-first-out (FIFO) principle, meaning that the first element added to the queue will be the first one removed. Queues are commonly used in computer science for managing and organizing data in a specific order.

• Some common applications of Queue :-

- (i) Task Scheduling
- (ii) Resource Allocation
- (iii) Batch Processing
- (iv) Event Handling
- (v) Traffic Management
- (vi) Web Servers
- (vii) Network Protocols.

Q3. What is Priority Queue and Dequeue?

Ans Priority Queue: A Priority Queue is a kind of queue in which each element has a priority associated with it. Element with higher priorities are served before element with lower priorities. Priority queues are often implemented using a heap data structure.

- Dequeue (Double-Ended Queue):- A dequeue is a double-ended queue, which means that elements can be added and removed from the front and the back of the queue. Dequeues are often implemented using a doubly linked list data structure.

Q4. Explain header Linked List.

Ans A header linked list is a type of linked list that uses a special header node to represent the beginning of the list. The header node is actually a dummy node with no actual data, but rather acts as a reference point to the first node in the list.

Header linked list are often used to implement abstract data types such as stacks, queues, and doubly linked lists. They can also be used to implement more complex data structures, such as polynomials and graphs.

Q5. Write limitations of Linked List.

Ans Linked lists are a powerful data structure, but they have some limitations.

- (i) Inefficient random access:- Linked lists are not efficient for random access to elements. To access an element in a linked list, you have to start at the beginning of the list and traverse through it until you find the desired element. This can be inefficient for large lists.

- (ii) Extra memory usage :- Linked lists require extra memory to store pointers to the next element in the list. This can be a significant overhead for large lists.
- (iii) Complex implementation :- Linked lists are more complex to implement than other data structures, such as arrays.
- (iv) Memory fragmentation -
- (v) Not suitable for storing large data sets.

• Set-B ($3 \times 7 = 21$)

Q5. Write the algorithm of insertion in the circular linked list.

Ans.

Ans

- (1) Create a new node with the data to be inserted
- (2) if the list is empty, set the new node to be the head node and the tail node.
- (3) Otherwise, insert the new node after the tail node.
- (4) Update the tail node's next pointer to point to the new node.
- (5) if the new node is being inserted at the beginning of the list, update the head node's pointer to point to the new node.

Algorithm :-

Step 1

if $Ptr = null$

write overflow

Go to step 11

[End of if]

Step-2

Set New-Node = Ptr

- Step 3 Set $ptr = ptr \rightarrow NEXT$
 Step 4 Set New Node $\rightarrow Data = val$
 Step 5 Set $temp = Head$
 Step 6 Repeat step 3 while $Temp \rightarrow NEXT \neq Head$
 Step 7 Set $temp = temp \rightarrow NEXT$
 [End of Loop]
 Step 8 Set New Node $\rightarrow NEXT = Head$
 Step 9 Set $temp \rightarrow NEXT = New Node$
 Step 10 Set $Head = New Node$
 Step 11 Exit

Q2. Write the algorithm of insertion in circular Queue.

Ans

- (1) Check if the Queue is Full. If the Queue is full, return an error message. Otherwise, proceed to step 2.
- (2) Increment the rear pointer by 1. If the rear pointer reaches the end of the Queue, wrap it around to the beginning.
- (3) Insert the new element at the rear of the Queue.
- (4) Check if the front pointer is -1. If it is, set it to 0.
- (5) If the rear pointer is equal to the front pointer, the Queue is now full.

Algorithm :

- Step 1 Start
 Step 2 Store element to insert in linear data
 Step 3 Check if $(front == 0 \text{ \& \& } rear = Max - 1) \parallel (front = rear + 1)$
 then Queue Overflow else go to Step 4.
 Step 4 Check if $(front == -1)$ then $front = 0$; $rear = 0$;
 else go to Step 5.

- Step 5 check if $(\text{rear} = \text{max} - 1)$ then $\text{rear} = 0$; else
 $\text{rear} = \text{rear} + 1$; and go to step 6.
- Step 6 insert element ($\text{queue} - \text{queue}[\text{rear}] = \text{item}$)
- Step 7 Stop.

Q3.

How can we use stack for implementation of Queue?
 Explain.

Ans.

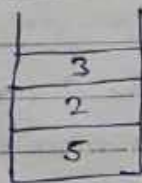
Suppose we have a queue.

enqueue (5),

enqueue (2),

enqueue (3),

Now queue would look like



In above stack, we can observe that topmost is 3. If we perform deletion operation then element 3 would be deleted. On other hand, deletion in Queue is performed from front end and front element is 5. In order to implement Queue using stack, we need to consider two stacks.

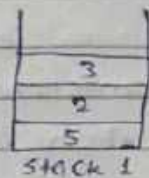


Stack 1

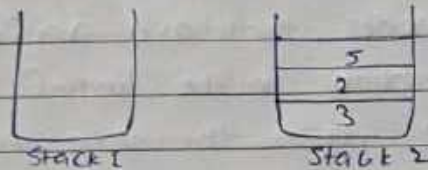


Stack 2

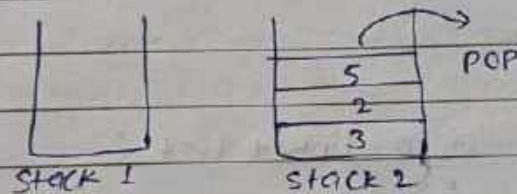
As we observe above stacks are empty, Now we perform Push operation on Stack 1.



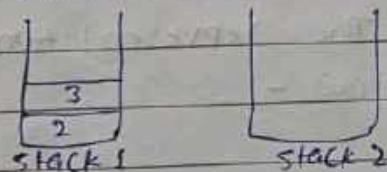
Now we will pop element from stack 1, one by one and push them in stack 2.



once element are inserted into stack 2, topmost element is 5 so, it could be popped out from stack 2.



once topmost element is popped out from stack 2 all element are moved back to stack 1.



There are two approaches to implement Queue using stack

- Making dequeue operation costly
- Making enqueue operation costly

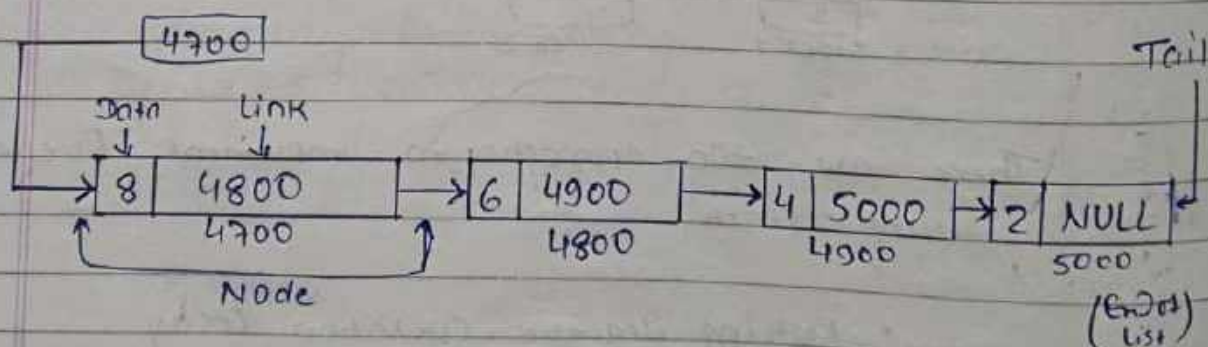
- Set-C ($3 \times 7 = 21$)
- Q1. Explain the memory representation of linked list & write the algorithm of deletion of a node from doubly linked list.

Ans

Linked list is a linear data structure that includes a series of connected nodes. Linked list can be defined as the nodes that are randomly stored in the memory. A node in the linked list contains two parts, i.e., first is the data part and second is the address part. The last node of the list contains a pointer to the null. After Array, linked list is the second most used data structure. In a linked list, every link contains a connection to another link.

- Representation of a linked list:

Linked list can be represented as the connection of nodes in which each ~~other~~ node points to the next node of the list. The representation of the linked list is shown below -



Till now, we have been using array data structure to organize the group of elements that are to be stored individually in the memory.

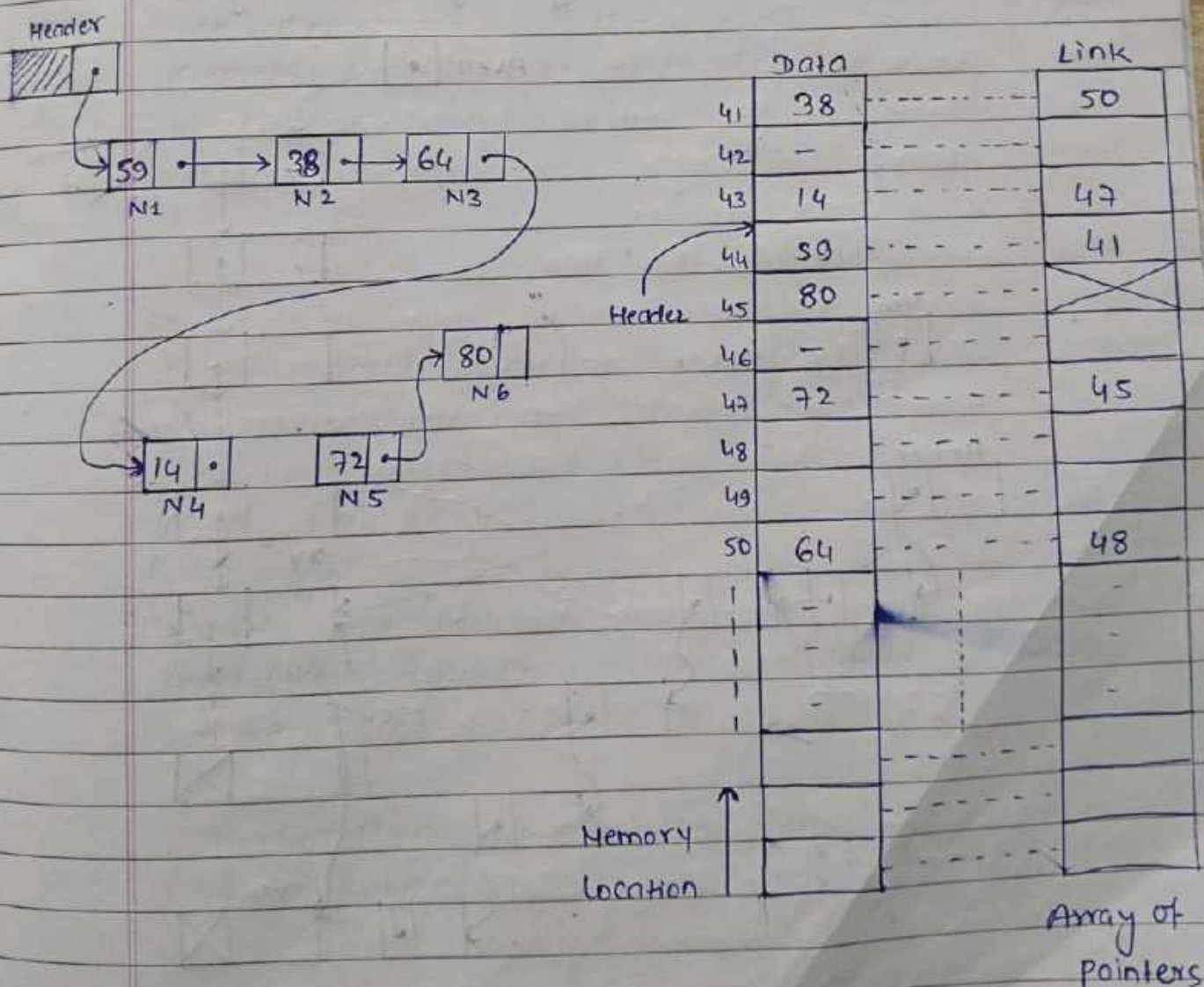
- Representation of a linked list in memory :

There are two way to represent a linked list in memory.

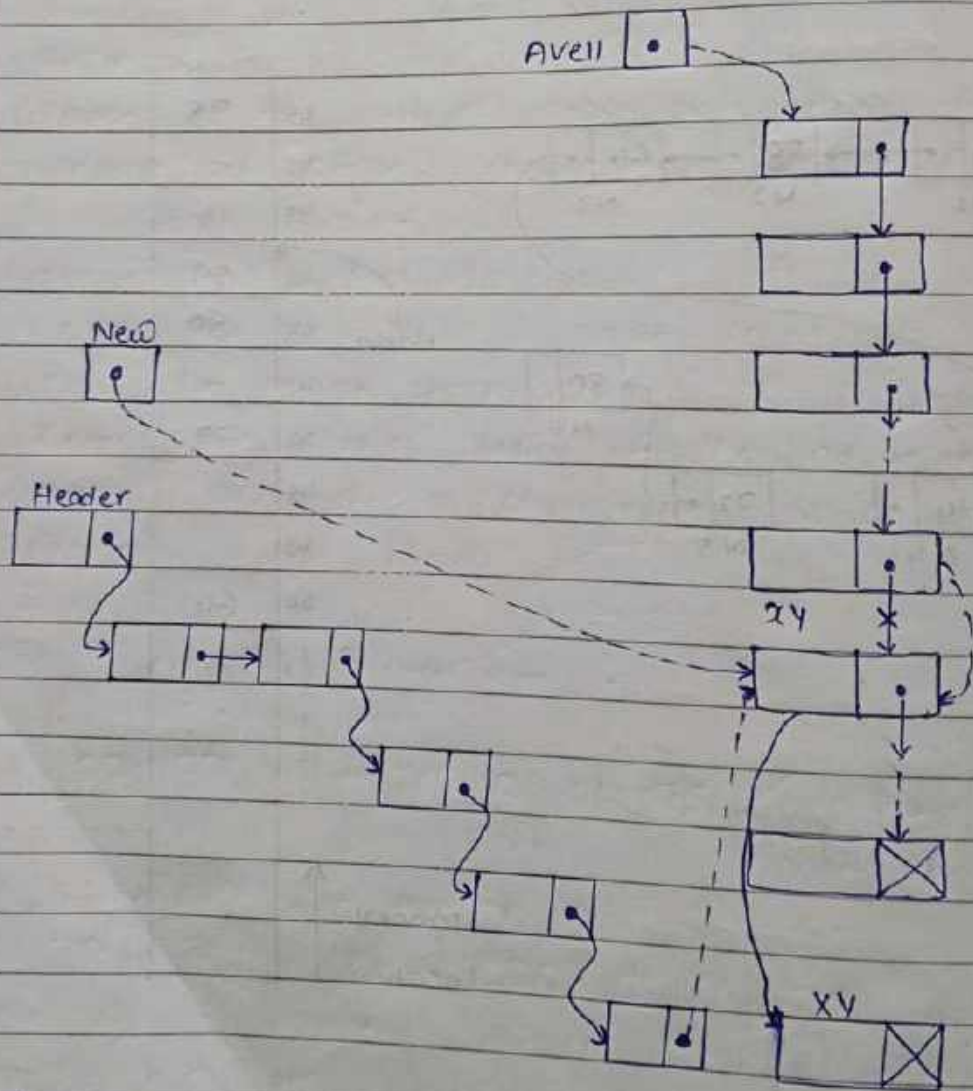
- (1) Static representation using array
- (2) Dynamic representation using free pool of storage.

- (2) Dynamic representation using free pool of storage.

(1) Static representation : In static representation of a single linked list, two arrays are maintained: one array for data and the other for links.



(2) Dynamic Representation: The efficient way of representing a linked list is using free pool of storage. In this method, there is a memory bank and a memory bank manager. During the creation of a linked list whenever a node is required the request is placed to the memory manager; the memory manager will then search the memory bank for the block requested and, if found, grants the desired block to the caller. The dynamic representation of linked list uses the dynamic memory management policy.



• (Allocation of a node from memory bank to a linked list)

• Algorithm of deletion of a node from Doubly Linked List:

- Step 1: if Head = NULL
write underflow
Go to step 9
[End of if]
- Step 2: Set temp = Head
- Step 3: Repeat step 4 while temp → Data != item
- Step 4: Set Temp = Temp → Temp NEXT
[End of loop]
- Step 5: Set Ptr = temp → NEXT
- Step 6: Set Temp → NEXT = Ptr → NEXT
- Step 7: Set Ptr → NEXT → PREV = temp
- Step 8: ~~set~~ Free Ptr
- Step 9: Exit;

Pseudocode:

```

public class DoublyLinkedList {
    private Node head;
    public DoublyLinkedList() {
        this.head = null;
    }
    public void deleteNode(Node nodeToDelete) {
        if (head == null) {
            return;
        }
        if (nodeToDelete == head) {
            head = head.next;
        }
        if (nodeToDelete.next == null) {
            nodeToDelete.prev.next = null;
        }
    }
}

```

```

nodeToDelete.prev.next = nodeToDelete.next;
nodeToDelete.next.prev = nodeToDelete.prev;

```

```

nodeToDelete = null;
}
}

```

Q2. Write the algorithm of Merge sort and apply on this (22, 33, 44, 11, 20, 12, 66, 55).

Ans.

• Algorithm of Merge sort:

Step 1: Begin

Step 2: Divides all the item of array into two items pairs. if single item remains make a single group.

Step 3: Sort each of the Pair.

Step 4: Merge two pairs into a single pairs.

Step 5: Sort the merge pair.

Step 6: Repeat the above step until the all elements of array are not sorted.

Step 7: exit;

• Apply Merge sort Algorithm:

22, 33, 44, 11, 20, 12, 66, 55

1. Splits the list into two Pairs:

22	33	44	11	20	12	66	55
----	----	----	----	----	----	----	----

2. Now Divide the left subList into smaller sub list:

22	33	44	11
----	----	----	----

5. Sort the elements in their appropriate positions and then combined the sub lists.

22	33
----	----

11	44
----	----

4. Now, these sublists are again merged to give the following sorted sub list of size 4.

11	22	33	44
----	----	----	----

5. Same process for the right sub list also.

20	12	66	55
----	----	----	----

20	12
----	----

66	55
----	----

12	20
----	----

55	66
----	----

12	20	55	66
----	----	----	----

6. Finally, the left and right halves of the array are merged to give the sorted array as follows.

11	12	20	22	33	44	55	66
----	----	----	----	----	----	----	----

sorted Array

- Q3. Write the algorithm of Quick sort and apply on this (22, 33, 44, 11, 20, 12, 66, 55).

Ans. Algorithm:

Step 1: Begin

Step 2: Select the start element of array as a pivot element.

Step 3: Scan and find smallest element from right side of array.

Step 4: Interchange both element.

Step 5: Scan and find the biggest element from left side of array.

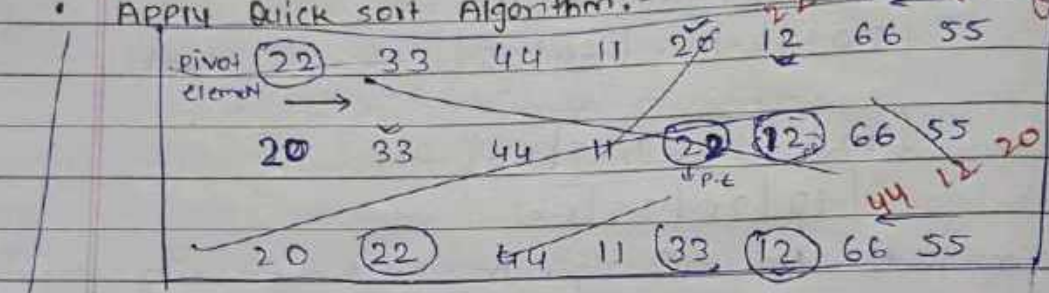
Step 6: Repeat the above process until all the elements

of left side are smaller and right side elements are greater than pivot element.

Step 7: Now, we have two sublists available, apply same process on each sublist until the all element of array are not sorted.

Step 8: exit.

• Apply Quick sort Algorithm:-



→ Pivot Element → 22 33 44 11 20 12 66 55
20 33 44 11 22 12 66 55
20 22 44 11 33 12 66 55
20 12 44 11 33 22 66 55
20 12 44 11 22 33 66 55

P.L. (20) 12 44 11
12 (20) 44 11
12 44 (20) 11
12 44 11 (20)
12 (20) 11 44
12 11 (20) 44
12 11
(11) 12
12

P.L. (22) 33 66 55
(66) 55
55 66

Jan 2, 2024, 17:50 | 11 12 20 22 33 44 55 66 → sorted Array =

Section - A

Q1. Why Merge sort a better option than quicksort for linked list?

Ans Merge sort is often favoured over quicksort for linked list due to several reason :-

- (i) Stability : Merge sort preserves the order of equal element important in certain scenarios.
- (ii) Predictable Performance : Merge sort doesn't depend on pivot selection, ensuring consistent performance.
- (iii) No need for Random access : Quicksort requires random access for efficient partitioning, which is inefficient in linked list.
- (iv) Worst-case time Complexity :- Merge sort maintains a consistent $O(n \log n)$ time complexity, whereas quicksort can degrade to $O(n^2)$ in worst case.

Q2. How will you remove cycle from a linked list?

Ans To remove a cycle from a linked list :

- (i) Use Floyd's cycle Detection Algorithm
- (ii) Detect the cycle using slow and fast pointers.
- (iii) Find the start of the cycle where the pointers meet.
- (iv) Break the cycle by making the node before the start point to null.

Q3. What is Dictionary data structure?

Ans A dictionary data structure is a collection of key-value pairs. Each key is unique and is used to identify the corresponding value. Dictionaries are fundamental data structure that provide a flexible and efficient way to store and organize data. Their ability to associate keys with values makes them invaluable for various programming tasks and real-world applications.

Q4. What are the benefits of a Priority Queues? Give 2 applications of it.

Ans. Benefits of Priority Queues:

- (i) Ordering by Priority: Allows storing elements based on priority for efficient retrieval.
- (ii) Efficient Operations: Offers quick insertion, deletion and access based on priority.

• Applications of Priority Queues:

- (i) Task scheduling and Process Management.
- (ii) Event handling and Interrupt Management.
- (iii) Job Queues and Resource Allocation.
- (iv) Data Streaming and Real-time Processing.

Q5. How can we reverse a single link list?

Ans. To reverse a singly linked list.

- (i) Initialize three pointers: 'prev', 'current', & 'next_node'.
- (ii) Traverse the list, reversing pointers:
 - Save 'next_node'.
 - Reverse the current node's pointer to 'prev'.
 - Move pointers to the next nodes.
- (iii) Update the head of the list to 'prev' when traversal complete.

• Section-B : (3x7=21)

Q1. Write an algorithm to implement a queue with the help of two stacks.

Ans. To implement queue using two stack, we can use one stack for enqueue operation and other for dequeue operation.

Algorithm.

- Create two empty stack, stack 1 and stack 2
- For enqueue operation
 - Push element onto stack 1
- For dequeue operation
 - If stack 2 is empty, transfer all elements from stack 1 to stack 2 until stack 1 is empty.
 - Pop top element from stack 2 and return it as dequeued element.
- If both stack are empty, queue is empty.

Q2 Write an algo. to delete nth node from a doubly linked list.

Ans

- If doubly linked list is empty, return
- If it has only one node and position to delete is 1, set head to null and return.
- If position to delete is 1, set head to next node and update previous pointer of new head to null
- traverse it to (n-1) node
- If position to delete is > no. of nodes in it. return
- set next pointer of (n-1)th node to null after nth node.
- If next node exists, update its previous pointer to (n-1)th node.
- Delete nth node by setting its next and previous pointers to null
- Return modified doubly linked list.

Q3 Write a program to implement linear queue using arrays.

Ans

```
public class DemoQueue {  
    private int maxSize;  
    private int[] queueArray;  
    private int front;  
    private int rear;  
    private int currentSize;  
    public DemoQueue (int size) {  
        this.maxSize = size;  
        this.queueArray = new int[size];  
        front = 0;  
        rear = -1;  
        currentSize = 0;  
    }
```

```
    public void insert (int item) {  
        if (isQueueFull()) {  
            System.out.println ("Queue is full");  
            return;  
        }
```

```
        if (rear == maxSize - 1) {  
            rear = -1;
```

```
        }  
        queueArray (++rear) = item;  
        currentSize ++;
```

```
        s.o.p ("item added to queue: " + item);  
    }
```

```
    public int delete () {
```

```
        if (isQueueEmpty ()) {
```

```
            throw new RuntimeException ("Queue is empty");
```

```
        }
```



```

int temp = queueArray[front + 1];
if (front == maxSize) {
    front = 0;
}
currentSize--;
return temp;
}

public int peek() {
    return queueArray[front];
}

public boolean isQueueFull() {
    return (maxSize == currentSize);
}

public boolean isQueueEmpty() {
    return (currentSize == 0);
}

public static void main (String [] args) {
    DemoQueue queue = new DemoQueue (10);
    queue.insert (2);
    System.out.println (" item deleted from queue " +
        queue.delete ());
}
}

```

Output :

item added to queue : 2
item delete from queue : 2

- Sec-C ($3 \times 11 = 33$)
- Q1. Write an algo. to remove all odd positioned nodes from a circular linked list considering the index starting from 1.

Ans. → Start at head of circular linked list.

→ Traverse the list, keeping track of position of each node.

→ If position of current node is odd, remove it from list.

→ Continue traversing until you reach end. Finally, update next pointer of last node to point to new head of modified list.

Explanation :-

- Check if linked list is empty, if it is return.
- Initialize temp and back pointer for traversal.
- Traverse list using temp. if current node is position at odd, perform deletion by updating back next and moving temp pointer.
- Move to next node ($\text{back} = \text{temp next}$)
- Move temp to node following next node
 $\text{temp} = \text{back.next}$
- if traversal reaches end of list, update temp pointer to null to stop further traversal.

Q2

What is sorted Link List? Write an algorithm for searching in a sorted link list.

Ans

A sorted linked list is a data structure where elements are arranged in a specific order. Each element in linked list contain value and a reference to next element.

Algorithm -

- (1) → Start at head of linked list.
- (2) → Set two pointer, start and end to beginning and end, respectively.
- (3) → Calculate middle index b/w start and end
$$\text{middle} = \frac{\text{start} + \text{end}}{2}$$

4. → Move middle pointer to middle index

5. → Compare value at middle pointer to targeted value.

~~6. →~~

→ If they are equal, element is found, Return

→ If value at middle > targeted, update end to index before middle index and go to step 3.

→ If value at middle < targeted, update start to index after middle index and go to step 3.

6. Repeat step 3-5 until target value is found or start becomes > end.

7. If target value is not found, it is not present in linked list.

Q3

Write a Program to Implement Quicksort. Show the steps to sort the following numbers:

25, 13, 7, 34, 56, 23, 13, 96, 14, 2

Ans

```
import java.util.*;
```

```
import java.util.*;
```

```
public class QuickSort {
```

```
    public static void main (String[] args) {
```

```
        int[] array = { 25, 13, 7, 34, 56, 23, 13, 96, 14, 2 };
```

```
        quickSort (array, 0, array.length - 1);
```

```
        System.out.println (Array.toString (array));
```

```
    }
```

```
    public static void quickSort (int[] array, int low, int high)
```

```
    {
```

```
        if (low < high) {
```

```
            int pivot = Partition (array, low, high);
```

```
            quickSort (array, low, pivot - 1);
```

```
            quickSort (array, pivot + 1, high);
```

```
        }
```

```
    }
```

```
    public static int Partition (int[] array, int low, int high)
```

```
    {
```

```
        int pivot = array [high];
```

```
        int i = low - 1;
```

```
        for (int j = low; j < high; j++) {
```

```
            if (array [j] <= pivot) {
```

```
                Jan 2, 2024, 17:50
```

```
                i++;
```



```

    }
}
swap(array, i+1, high);
return i+1;
}

public static void swap(int[] array, int i, int j) {
    int temp = array[i];
    array[i] = array[j];
    array[j] = temp;
}
}

```

output :- { 2, 7, 13, 13, 14, 23, 25, 34, 56, 96 }

steps

{ 25, 13, 7, 34, 56, 23, 13, 96, 14, 2 }

partition around 2

{ 2 } (25, 13, 7, 34, 56, 23, 13, 96, 14)

partition around 14

(13, 7, 13) { 14 } { 25, 34, 56, 23, 96 }

partition around 13

partition around 96

{ 7 } { 13, 13 }

{ 25, 34, 56, 23 } { 96 }

partition around 23

{ 23 } { 25, 34, 56 }

partition around 56

{ 25, 34 } { 56 }

partition around 34

{ 25 } { 34 }

sorted array :-

{ 2, 7, 13, 13, 14, 23, 25, 34, 56, 96 }